

Code Explanation

Unit Tests for Customer Order Pipeline

This code provides a comprehensive set of unit tests for a customer order pipeline using PySpark. The tests cover various aspects of the pipeline, including data validation, data cleaning, total amount calculation, and customer aggregate spend calculation.

Overview of the Code

The code is written in Python and utilizes the PySpark library for data processing. It includes several test functions that verify the correctness of different components of the customer order pipeline. The tests are designed to ensure that the pipeline functions as expected and handles various edge cases.

Step 1: Setting up the Spark Session

The first step is to create a Spark session, which is a fundamental component of PySpark. This session is used throughout the tests to create DataFrames and perform data processing.

```
@pytest.fixture(scope="session")
def spark():
    return SparkSession.builder
        .appName("TestCustomerOrderPipeline")
        .master("local[1]")
        .getOrCreate()
```

Step 2: Defining Schemas for Customer and Order Data

The next step is to define the schemas for the customer and order data. These schemas specify the structure of the data, including the column names and data types.

```
@pytest.fixture
def customer_schema():
    return StructType([
        StructField("CustId", StringType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True)
    ])

@pytest.fixture
def order_schema():
    return StructType([
        StructField("OrderId", StringType(), True),
        StructField("ItemName", StringType(), True),
        StructField("PricePerUnit", DoubleType(), True),
        StructField("Qty", IntegerType(), True),
        StructField("Date", DateType(), True),
        StructField("CustId", StringType(), True)
    ])
```

Step 3: Creating Sample Customer and Order Data

The code creates sample customer and order data using the defined schemas. This data is used to test

the various components of the pipeline.

```
@pytest.fixture
def sample_customer_data(spark, customer_schema):
    data = [
        ("C001", "John Doe", "john@example.com", "North"),
        ("C002", "Jane Smith", "jane@example.com", "South"),
        ("C003", "Bob Johnson", "bob@example.com", "East"),
        ("C004", None, "alice@example.com", "West"),
        ("C005", "Tom Brown", "tom@example.com", "North"),
        ("C005", "Tom Brown", "tom@example.com", "North")
    ]

    return spark.createDataFrame(data, customer_schema)

@pytest.fixture
def sample_order_data(spark, order_schema):
    data = [
        ("0001", "Laptop", 1200.0, 1, datetime.date(2023, 1, 15), "C001"),
        ("0002", "Phone", 800.0, 2, datetime.date(2023, 1, 20), "C002"),
        ("0003", "Tablet", 500.0, 1, datetime.date(2023, 1, 25), "C003"),
        ("0004", "Headphones", 100.0, 3, datetime.date(2023, 1, 30), "C001"),
        ("0005", "Charger", 25.0, 5, datetime.date(2023, 2, 5), "C002"),
        ("0006", "Case", None, 2, datetime.date(2023, 2, 10), "C003"),
        ("0007", "Screen", 50.0, 1, datetime.date(2023, 2, 15), "C005"),
        ("0007", "Screen", 50.0, 1, datetime.date(2023, 2, 15), "C005")
    ]

    return spark.createDataFrame(data, order_schema)
```

Step 4: Testing Data Validation

The `test\_validate\_data` function tests the `validate\_data` function by passing in valid and invalid columns.

```
def test_validate_data(spark, sample_customer_data):
    assert validate_data(sample_customer_data, ["CustId", "Name", "EmailId", "Region"]) == True

    with pytest.raises(ValueError):
        validate_data(sample_customer_data, ["CustId", "Name", "NonExistentColumn"])
```

Step 5: Testing Data Cleaning

The `test\_data\_cleaning` function tests the data cleaning logic by filtering out rows with null values and duplicates.

```
def test_data_cleaning(spark, sample_customer_data, sample_order_data):
    cleaned_customer = sample_customer_data.filter(
        sample_customer_data.CustId.isNotNull() &
        sample_customer_data.Name.isNotNull() &
        sample_customer_data.EmailId.isNotNull() &
        sample_customer_data.Region.isNotNull()
    ).dropDuplicates(["CustId"])

    assert cleaned_customer.count() == 4

    cleaned_order = sample_order_data.filter(
        sample_order_data.OrderId.isNotNull() &
        sample_order_data.ItemName.isNotNull() &
        sample_order_data.PricePerUnit.isNotNull() &
        sample_order_data.Qty.isNotNull() &
        sample_order_data.Date.isNotNull() &
        sample_order_data.CustId.isNotNull()
    ).dropDuplicates(["OrderId"])

    assert cleaned_order.count() == 5
```

Step 6: Testing Total Amount Calculation

The `test\_total\_amount\_calculation` function tests the total amount calculation by verifying the results for specific orders.

```
def test_total_amount_calculation(spark, sample_order_data):
    from pyspark.sql.functions import col

    orders_with_total = sample_order_data.withColumn("TotalAmount",
        col("PricePerUnit") * col("Qty"))

    order_o001 = orders_with_total.filter(col("OrderId") == "0001").first()
    assert order_o001["TotalAmount"] == 1200.0

    order_o002 = orders_with_total.filter(col("OrderId") == "0002").first()
    assert order_o002["TotalAmount"] == 1600.0
```

Step 7: Testing Customer Aggregate Spend Calculation

The `test\_customer\_aggregate\_spend` function tests the customer aggregate spend calculation by verifying the results for a specific customer.

```
def test_customer_aggregate_spend(spark, sample_customer_data, sample_order_data):
    from pyspark.sql.functions import col, sum as sum_

    cleaned_customer = sample_customer_data.filter(
        sample_customer_data.CustId.isNotNull() &
        sample_customer_data.Name.isNotNull()
    ).dropDuplicates(["CustId"])

    cleaned_order = sample_order_data.filter(
```

```

        sample_order_data.OrderId.isNotNull() &
        sample_order_data.PricePerUnit.isNotNull()
    ).withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))

customer_spend = (
    cleaned_order
    .join(cleaned_customer, "CustId", "inner")
    .groupBy("Name", "Date")
    .agg(sum_("TotalAmount").alias("TotalAmount"))
)

john_spend = customer_spend.filter(col("Name") == "John Doe").collect()
assert len(john_spend) == 2

john_jan15 = [row for row in john_spend if row["Date"] == datetime.date(2023, 1, 15)]
assert len(john_jan15) == 1
assert john_jan15[0]["TotalAmount"] == 1200.0

```